

Manuale Operativo e Guida agli Aggiornamenti

Sito Web *Medical Physics & Complex Systems* (UniBa / INFN Bari)

Questo manuale è la guida completa per la comprensione, la gestione e l'aggiornamento ordinario e straordinario del sito istituzionale del gruppo **Medical Physics and Complex Systems** (Università degli Studi di Bari Aldo Moro · INFN Sezione di Bari).

Indice dei Contenuti

1. [Architettura e Funzionamento del Sito](#)
2. [Mappa dei File del Progetto](#)
3. [Ruolo degli Script Python in `tools/`](#)
4. [Regole Fondamentali prima di Modificare i Dati](#)
5. [Guida Pratica agli Aggiornamenti Ordinari](#)
 - [5.1 Aggiungere o modificare una Persona \(Staff / Collaboratori\)](#)
 - [5.2 Aggiungere o modificare un Progetto \(Ongoing / Past\)](#)
 - [5.3 Aggiungere o modificare Pubblicazioni](#)
 - [5.4 Aggiungere o modificare una Linea di Ricerca](#)
 - [5.5 Aggiungere o modificare una Sfida \(Challenge\)](#)
 - [5.6 Aggiornare le Notizie in Home \(Newsroom\)](#)
 - [5.7 Modificare Pagine di Testo Lungo / Pagine Istituzionali](#)
 - [5.8 Aggiornare Tesi di Laurea \(Join Us\) e Corsi](#)
 - [5.9 Modificare Dati Globali \(Brand, Email, Indirizzo, Footer\)](#)
6. [Estendere il Sito \(Nuove Voci di Menu e Nuove Pagine\)](#)
7. [Sicurezza e Best Practice \(CSP, Sanitizer, Immagini, Video\)](#)
8. [Test in Locale e Pubblicazione Online \(Deploy\)](#)
9. [Guida alla Risoluzione dei Problemi \(Troubleshooting\)](#)

1. Architettura e Funzionamento del Sito

Il sito è una **Single Page Application (SPA)** completamente statica:

- **Nessun framework pesante** (niente React, Angular, Vue o Next.js).
- **Nessun backend dinamico** (nessun server PHP, Node.js o Python in esecuzione continua).
- **Nessun database SQL**: tutti i contenuti risiedono in strutture dati JavaScript.
- **Routing basato sull'ancora URL (`location.hash`)**: la navigazione avviene tramite frammenti URL come ad esempio `#/home` , `#/people` , `#/research` , `#/projects` .

Come si genera una pagina a runtime

Nel browser, `index.html` fornisce un unico contenitore: `<div id="app"></div>` . All'avvio e a ogni variazione dell'URL:

1. `assets/js/app.js` legge l'hash corrente (es. `#/mario-rossi`).
2. Individua i dati corrispondenti interrogando le variabili globali caricate in memoria:
 - `window.MEDPHYS_SITE` (struttura, persone, progetti, menu, parametri globali)
 - `window.MEDPHYS_EN` e `window.MEDPHYS_META` (corpi delle pagine lunghe e metadati)
 - `window.MEDPHYS_IT` e `window.MEDPHYS_EN_X` (traduzioni e override)
3. Passa il markup attraverso il filtro di sicurezza `window.MedPhysSanitize` (`assets/js/sanitize.js`).
4. Genera il codice HTML completo (Header, Contenuto, Footer) e lo inserisce nel contenitore `#app` .
5. Imposta dinamicamente il titolo della scheda nel browser (`document.title`).

`index.html`

```
| (Carica i seguenti script nell'ordine esatto)
|→ 1. assets/data/site-data.js    → window.MEDPHYS_SITE
|→ 2. assets/data/pages_en.js    → window.MEDPHYS_EN e window.MEDPHYS_META
|→ 3. assets/data/i18n.js        → window.MEDPHYS_IT e window.MEDPHYS_EN_X
|→ 4. assets/js/sanitize.js      → window.MedPhysSanitize (filtro XSS obbligatorio)
|→ 5. assets/js/app.js           → Router, renderer, logica multilingua e layout
```

2. Mappa dei File del Progetto

```
MedPhys_Sito/
├─ index.html          # Guscio HTML principale (carica CSS e i 5 script JS)
├─ robots.txt          # Istruzioni per i motori di ricerca
├─ README.md           # Documentazione sintetica del repository
├─ manuale_def.md       # Questo manuale operativo
├─ vulnerabilita.md     # Registro e storico dei controlli di sicurezza
├─
├─ assets/
│   ├─ css/
│   │   └─ style.css    # Foglio di stile unico (variabili colore, layout, responsive)
│   ├─ js/
│   │   ├─ sanitize.js  # Sanitizer HTML contro attacchi XSS
│   │   └─ app.js       # Router client-side e motori di rendering
│   ├─ data/            # ← DOVE RISIEDONO I CONTENUTI DEL SITO
│   │   ├─ site-data.js # Struttura generale, menu, staff, progetti, news
│   │   ├─ pages_en.js  # Testi HTML lunghi in inglese e metadati
│   │   ├─ i18n.js       # Traduzioni e testi italiani
│   │   ├─ pages_en.json # Copia JSON di backup/sorgente di pages_en.js
│   │   └─ meta.json     # Copia JSON dei titoli e metadati
│   └─ img/             # Immagini, loghi, foto dello staff (~250 file)
├─
├─ tools/              # Script Python ausiliari (vedi Sezione 3)
│   ├─ gen_en.py        # Compilatore facoltativo da JSON a JS (NON usare di norma)
│   ├─ build_site_data.py # Script di scraping/migrazione iniziale dal vecchio sito
│   ├─ build_i18n.py     # Script di migrazione traduzioni
│   ├─ clean_content.py  # Pulizia dell'HTML ereditato
│   ├─ download_images.py # Download delle immagini originali
│   ├─ security.test.cjs # Test automatici di sicurezza
│   ├─ render.test.cjs   # Test automatici di rendering
│   └─ test_import_pipeline.py # Test della pipeline di importazione
├─
└─ .github/
    └─ workflows/
        └─ pages.yml     # Pipeline di pubblicazione su GitHub Pages
```

3. Ruolo degli Script Python in `tools/`

Important

I file Python in `tools/` **NON** servono al funzionamento del sito e **NON** devono essere eseguiti per i normali aggiornamenti.

- **Al runtime:** Il sito è 100% statico nel browser; non esiste alcun processo Python in ascolto.
- **Perché esistono:** Sono script creati *una tantum* per migrare i contenuti dal vecchio portale WordPress/Drupal e ripulire l'HTML.
- **Devo eseguire `gen_en.py` se cambio una pagina, un progetto o una persona?**
NO, ASSOLUTAMENTE NO.
 - I dati di produzione risiedono direttamente nei file `.js` in `assets/data/`.
 - Lo script `gen_en.py` legge dal file statico intermedio `pages_en.json` e **sovrascrive inesorabilmente** `pages_en.js`. Se lanciassi `gen_en.py`, cancelleresti tutte le modifiche manuali apportate a `pages_en.js` !

L'unico motivo per cui i file in `tools/` restano nel repository è per memoria storica e perché la suite di test automatici di GitHub Actions (`[pages.yml](file:///github/workflows/pages.yml#L37)`) ne verifica l'integrità come gate di sicurezza prima della pubblicazione.

4. Regole Fondamentali prima di Modificare i Dati

Prima di aprire e modificare i file in `assets/data/`, tieni a mente queste 3 regole pratiche:

1. **Non cercare pagine `.html` per le varie sezioni:**

Non esistono `people.html`, `projects.html` o `publications.html`. Tutto viene generato dai dati JavaScript.

2. **Usa la formattazione automatica in VS Code:**

I file `.js` in `assets/data/` sono compressi su poche righe. Quando ne apri uno in Visual Studio Code:

- Premi **Shift + Alt + F** (su Windows/Linux) oppure **Shift + Option + F** (su Mac) per indentare il codice e renderlo leggibile.
- Prima di salvare, controlla con attenzione di non aver dimenticato virgole o parentesi graffe.

3. **Forza l'aggiornamento della cache del browser:**

Dopo aver salvato un file `.js`, ricarica la pagina nel browser con un **Hard Refresh**:

- **Windows/Linux:** Ctrl + F5 (oppure Ctrl + Shift + R)
- **Mac:** Cmd + Shift + R

Altrimenti il browser continuerà a mostrare la versione memorizzata in cache.

5. Guida Pratica agli Aggiornamenti Ordinari

Tutti gli aggiornamenti descritti di seguito si effettuano modificando direttamente i file in `assets/data/` .

5.1 Aggiungere o modificare una Persona (Staff / Collaboratori)

- **File target:** `[ assets/data/site-data.js ](file:///assets/data/site-data.js)`
- **Blocco:** array `"people"` (per lo staff visualizzato nella griglia di `#/people`) oppure `"collaborators"` (per i collaboratori esterni).

Modello da inserire:

```
{
  "slug": "mario-rossi",
  "img": "assets/img/mario_rossi.jpg",
  "keywords": {
    "en": "Medical Imaging, Deep Learning, Complex Systems",
    "it": "Imaging Medico, Deep Learning, Sistemi Complessi"
  },
  "bio": {
    "en": "Mario Rossi is a Postdoctoral Researcher at INFN Bari...",
    "it": "Mario Rossi è un ricercatore post-doc presso l'INFN di Bari..."
  },
  "email": "mario.rossi@uniba.it"
}
```

Regole importanti:

- **slug** : deve essere obbligatoriamente in minuscolo con trattini (nome-cognome). La funzione `nameOf()` converte automaticamente `mario-rossi` nel titolo "Mario Rossi".
- **img** : carica preventivamente l'immagine nella cartella `assets/img/` (preferibilmente proporzione quadrata 1:1).

- **keywords** : separa le voci con virgole; ciascun termine diventerà automaticamente un badge colorato nella scheda personale.
- **Raggiungibilità automatica**: non serve creare alcuna pagina; il router renderà subito accessibile la scheda dettagliata all'URL `#/mario-rossi` .

5.2 Aggiungere o modificare un Progetto (Ongoing / Past)

- **File target**: `[ assets/data/site-data.js ](file:///assets/data/site-data.js)`
- **Blocco**: `"projects"` → array `"ongoing"` (progetti attivi) o `"past"` (progetti conclusi).

Modello da inserire:

```
{
  "slug": "progetto-deepmed",
  "img": "assets/img/logo_deepmed.png",
  "title": {
    "en": "DEEPMED - AI in Clinical Diagnostics",
    "it": "DEEPMED - IA nella Diagnostica Clinica"
  },
  "duration": "Gen 2026 - Dic 2028",
  "period": "",
  "abstract": {
    "en": "Development of novel deep learning pipelines for neuroimaging biomarkers.",
    "it": "Sviluppo di modelli innovativi di deep learning per biomarcatori di neuroimaging."
  }
}
```

Lo stato ("Ongoing" o "Completed") viene assegnato in automatico dal sistema a seconda che l'oggetto sia inserito in `ongoing` o `past` .

5.3 Aggiungere o modificare Pubblicazioni

Le pubblicazioni sono gestite a due livelli:

Livello A: Nuova categoria di pubblicazioni

Se desideri aggiungere una nuova tipologia (es. articoli divulgativi):

- **File target:** [assets/data/site-data.js](file:///assets/data/site-data.js) → blocco "publications" .

```
{
  "slug": "popular-science",
  "en": "Popular science articles",
  "it": "Articoli divulgativi",
  "desc": {
    "en": "Outreach and science communication articles.",
    "it": "Articoli di divulgazione scientifica e disseminazione."
  }
}
```

Livello B: Inserire un nuovo articolo in una categoria esistente

I singoli articoli si inseriscono direttamente nel corpo HTML della categoria corrispondente:

- **File target:** [assets/data/pages_en.js](file:///assets/data/pages_en.js)
- **Blocco:** window.MEDPHYS_EN["journal-papers"] (o "book-chapters" , "refereed-proceedings").
- **Formato:**

```
<p><em>Titolo Completo dell'Articolo in Corsivo.</em></p>
<p>Rossi M., Bianchi L., Bellotti R.</p>
<p>Journal of Medical Physics, 45(2), 120-135, 2026.</p>
<p><a href="https://doi.org/10.xxxx/xxxx" target="_blank" rel="noopener noreferrer">doi:10.xxxx,
```

Note

Se desideri che i titoli compaiano tradotti o personalizzati in italiano, inserisci lo stesso slug e il contenuto in lingua italiana dentro window.MEDPHYS_IT["journal-papers"] nel file [assets/data/i18n.js](file:///assets/data/i18n.js).

5.4 Aggiungere o modificare una Linea di Ricerca

- **File target:** [assets/data/site-data.js](file:///assets/data/site-data.js)
- **Blocco:** array "research" .

Modello da inserire:

```
{
  "slug": "quantum-imaging",
  "img": "assets/img/quantum_imaging.jpg",
  "title": {
    "en": "Quantum Imaging",
    "it": "Imaging Quantistico"
  },
  "summary": {
    "en": "Application of quantum optics protocols to biomedical imaging.",
    "it": "Applicazione di protocolli di ottica quantistica all'imaging biomedico."
  },
  "body": {
    "en": "<p>Extended English text explaining the research topic in detail...</p>",
    "it": "<p>Testo esteso in italiano che descrive nel dettaglio la linea di ricerca...</p>"
  }
}
```

Tip

Se il campo `body` viene omesso, il sistema cercherà come fallback il contenuto testuale presente in `window.MEDPHYS_EN[slug]` nel file `pages_en.js`.

5.5 Aggiungere o modificare una Sfida (Challenge)

- **File target:** `[ assets/data/site-data.js ](file:///assets/data/site-data.js)`
- **Blocco:** array `"challenges"`.

```
{
  "slug": "isic-melanoma-2026",
  "img": "assets/img/challenge_isic.png",
  "title": {
    "en": "ISIC Skin Lesion Analysis",
    "it": "Analisi Lesioni Cutanee ISIC"
  },
  "desc": {
    "en": "International competition on skin cancer detection.",
    "it": "Competizione internazionale per l'identificazione precoce del melanoma."
  },
  "url": "https://challenge.isic-archive.com/"
}
```

Il campo `url` alimenta il pulsante di reindirizzamento esterno verso la piattaforma della competizione.

5.6 Aggiornare le Notizie in Home (Newsroom)

I loghi delle testate giornalistiche che parlano del gruppo compaiono nella sezione finale della Home.

- **File target:** [`assets/data/site-data.js`](`file:///assets/data/site-data.js`)
- **Blocco:** array `"newsroom"` .

```
{
  "name": "Corriere del Mezzogiorno",
  "img": "assets/img/corriere.png",
  "url": "https://corrieredelmezzogiorno.corriere.it/..."
}
```

L'ordine nell'array stabilisce la sequenza di visualizzazione.

5.7 Modificare Pagine di Testo Lungo / Pagine Istituzionali

Pagine come `#/multimedia` , `#/privacy-cookie-policy` o demo interattive non hanno schede rigide, ma visualizzano HTML formattato.

Per modificarle o crearne una nuova:

1. **Contenuto inglese / predefinito:** Apri [assets/data/pages_en.js](file:///assets/data/pages_en.js) e modifica il valore della chiave corrispondente in window.MEDPHYS_EN :

```
window.MEDPHYS_EN["multimedia"] = "<p>Testo o markup...</p>";
```

2. **Titolo della pagina:** Nello stesso file, aggiorna window.MEDPHYS_META :

```
window.MEDPHYS_META["multimedia"] = { "title": "Multimedia Gallery", "link": "", "fn": "" }
```

3. **Versione italiana:** Se vuoi fornire il testo in italiano, aprilo in [assets/data/i18n.js] (file:///assets/data/i18n.js) dentro window.MEDPHYS_IT :

```
window.MEDPHYS_IT["multimedia"] = "<p>Testo in italiano...</p>";
```

5.8 Aggiornare Tesi di Laurea (Join Us) e Corsi

- **File target:** [assets/data/site-data.js](file:///assets/data/site-data.js)

Tesi di Laurea (join_us):

```
"theses": [
  {
    "t": {
      "en": "Graph Neural Networks for Brain Connectomics",
      "it": "Graph Neural Networks per la Connettistica Cerebrale"
    },
    "s": "Prof. Roberto Bellotti, Dott. Nicola Amoroso"
  }
]
```

Corsi Universitari (courses):

L'elenco dei corsi tenuti dal gruppo si aggiorna nell'array courses.items :

```
"items": [
  {
    "en": "Pattern Recognition – Prof. Roberto Bellotti",
    "it": "Pattern Recognition – Prof. Roberto Bellotti"
  }
]
```

5.9 Modificare Dati Globali (Brand, Email, Indirizzo, Footer)

Tutti i parametri istituzionali generali sono definiti all'inizio di [`assets/data/site-data.js`] (`file:///assets/data/site-data.js`):

Proprietà	Descrizione
"brand"	Nome per esteso dell'istituto o gruppo di ricerca
"short"	Nome abbreviato impiegato negli occhielli e nei titoli compatti
"org"	Dipartimento o ente di appartenenza
"logo"	Percorso del file del logo (es. <code>assets/img/MedicalPhysics.png</code>)
"address"	Indirizzo fisico completo dei laboratori/sede
"emails"	Array degli indirizzi email ufficiali di contatto

I testi di copyright e note legali del footer sono generati in `assets/js/app.js:61-88` (l'anno corrente viene calcolato in automatico).

6. Estendere il Sito (Nuove Voci di Menu e Nuove Pagine)

Per creare una nuova voce nel menu di navigazione in alto (che si riflette in automatico anche nel footer e nel menu mobile):

1. Apri [`assets/data/site-data.js`](`file:///assets/data/site-data.js`).
2. Individua l'array `"nav"` .
3. Aggiungi il nuovo elemento:

Caso 1: Voce di menu singola (link diretto)

```
{
  "route": "collaborations",
  "en": "Collaborations",
  "it": "Collaborazioni"
}
```

Caso 2: Voce di menu a tendina (Dropdown con sottovoci)

```
{
  "route": "resources",
  "en": "Resources",
  "it": "Risorse",
  "children": [
    { "route": "datasets", "en": "Open Datasets", "it": "Dataset Pubblici" },
    { "route": "software", "en": "Software & Tools", "it": "Software e Tool" }
  ]
}
```

4. Definisci il contenuto della pagina inserendo lo slug (es. `datasets`) in `assets/data/pages_en.js` :

```
window.MEDPHYS_EN["datasets"] = "<h2>Open Datasets</h2><p>Elenco dataset...</p>";
window.MEDPHYS_META["datasets"] = { "title": "Open Datasets", "link": "", "fn": "" };
```

La nuova pagina sarà immediatamente attiva e navigabile con URL `#/datasets` .

7. Sicurezza e Best Practice (CSP, Sanitizer, Immagini, Video)

Il sito adotta una rigida **Content Security Policy (CSP)** definita in `index.html` e un componente di sanificazione in `assets/js/sanitize.js` . Per non rischiare che i browser blocchino contenuti:

1. Niente gestori JavaScript inline:

Non inserire mai `onclick="..."` , `onload="..."` o simili nell'HTML. Vengono rimossi dal sanitizer e bloccati dalla CSP.

2. **Niente attributi `style="..."` inline:**

La formattazione va affidata alle classi CSS già definite in `assets/css/style.css` (o a nuove classi lì aggiunte).

3. **Link esterni solo su HTTPS:**

I link verso l'esterno devono sempre iniziare per `https://` ed è buona norma includere `target="_blank" rel="noopener noreferrer"`.

4. **Immagini solo locali:**

Non collegare immagini ospitate su domini esterni. Salva sempre il file in `assets/img/` e inserisci il percorso relativo: `assets/img/nome_file.jpg`.

5. **Video YouTube e Privacy:**

La CSP ammette per i video unicamente il dominio `privacy-friendly` di YouTube:

- Host ammesso: `https://www.youtube-nocookie.com/embed/...`
- Wrapper richiesto per il layout responsive:

```
<div class="video-embed">
  <iframe src="https://www.youtube-nocookie.com/embed/CODICE_VIDEO"
    title="Video"
    frameborder="0"
    allowfullscreen>
  </iframe>
</div>
```

8. Test in Locale e Pubblicazione Online (Deploy)

Come visualizzare il sito sul proprio computer

Non serve installare Node.js, né Apache, né database:

1. **Metodo diretto:** Fai doppio clic su `[index.html]`(`file:///index.html`). Si aprirà direttamente nel browser.
2. **Metodo server locale** (consigliato per simulare le condizioni di rete):
Apri il terminale nella cartella del progetto ed esegui:

```
python -m http.server 8000
```

Quindi naviga su `http://localhost:8000`.

Come funziona il Deploy (GitHub Pages)

Il deploy è completamente automatizzato tramite GitHub Actions ([.github/workflows/pages.yml](file:///github/workflows/pages.yml)):

1. A ogni comando `git push` sul branch `main` :

- Vengono eseguiti i controlli di sicurezza (`security.test.cjs`) e di rendering (`render.test.cjs`).
- Viene generata una cartella di pubblicazione pulita (`_site`) che contiene **solo** `index.html` , `assets/` , `robots.txt` e la cartella `.well-known` .
- La cartella `tools/` e i file markdown interni vengono rigorosamente esclusi per ragioni di sicurezza e pulizia.
- Il sito compilato viene pubblicato all'URL di produzione.

9. Guida alla Risoluzione dei Problemi (Troubleshooting)

Il sito mostra una pagina completamente bianca

- **Causa:** Quasi certamente c'è un errore di sintassi JavaScript o JSON in uno dei file modificati (es. una virgola mancante, una parentesi graffa in più, o un apice non chiuso).
- **Soluzione:**
 - i. Apri la pagina nel browser e premi `F12` per aprire gli Strumenti per Sviluppatori.
 - ii. Guarda la scheda **Console**: troverai in rosso il file e il numero esatto di riga dove risiede l'errore di sintassi.
 - iii. Apri il file in VS Code, premi `Shift + Alt + F` per evidenziare la discrepanza, correggila e salva.

Ho modificato un file ma nel browser non cambia nulla

- **Causa:** Il browser ha mantenuto in memoria la copia precedente dello script JavaScript.
- **Soluzione:** Premi `Ctrl + F5` (Windows/Linux) o `Cmd + Shift + R` (Mac) per forzare il download dei file aggiornati.

Un'immagine appena inserita non si vede

- **Causa 1:** Mancata corrispondenza tra maiuscole e minuscole (es. `.JPG` invece di `.jpg`). Sui server Linux (come GitHub Pages) le maiuscole fanno la differenza.
- **Causa 2:** Percorso non corretto. Ricorda che il percorso deve essere relativo alla radice del sito:
`assets/img/nome-file.png` .

Un video YouTube resta un riquadro vuoto o bloccato

- **Causa:** L'URL dell'iframe usa `youtube.com` anziché `youtube-nocookie.com` .
- **Soluzione:** Sostituisci il dominio del link con `https://www.youtube-nocookie.com/embed/...` .